8 Classes and Objects 1

Viope instruction: Exercises with multiple source files

1. Copy/paste the source code of your program class to the Viope code editor window.
   * If there is a **package** directive, then remove it in Viope.

2. Copy/paste the other class definition to the Viope code editor window below the program class code.
   * Remove the **public** keyword from the class definition. Viope allows only the program class to be public.

3. If you have any ***import*** directives in your source texts, you have to move them to the top of the Viope editor window.
   * Place them before the program class code.

IMPORTANT!

- ✓ Please study the course materials carefully ***before asking for advice***. Thanks!
- ✓ The main objectives of this set of simplified exercises are to *familiarize yourself to object-oriented programming ideas* and to *strengthen your basic skills in creating program logic*. Therefore, it is important that you (1) ***read the exercise texts carefully*** and (2) ***follow carefully the instruction given in the exercise texts***.

Objects Song Program

First, write a new class called **Song** as described in the diagram below. You should not add any other features to the *Song* class. Note: The *Song* class should not contain any code for inputting data from the user or printing data.

Song
- String name - String artist - int year
+ Song(String name, String artist, int year) + String getName() + String getArtist() + int getYear() + String toString()

Then, create a main program called **ObjectsSongProgram**. In the program, create three objects from the *Song* class and assign their references to reference variables. See the example output for the song names, artists and years. **NOTE!** There should be *exactly three variables* in the main program. Not less, not more. This time, you should not use an array or ArrayList.

Finally, the program should print a song list as shown in the example output. In this exercise, use the ***toString*** method to get data from each *Song* object to be printed.

Example output:

```
Easy on Me (Adele, 2021)
Shivers (Ed Sheeran, 2021)
Holy Ghost Fire (Larkin Poe, 2020)
```

Objects Word Pairs Program

First, write a new class called **WordPair**. The features of the *WordPair* class should be exactly as shown in the diagram below. NOTE! The class should have no other fields or methods than shown in the diagram. The *WordPair* class should not contain any code for inputting data from the user or printing data.

```
WordPair
- String englishWord
- String finnishWord
+ WordPair(String englishWord, String finnishWord)
+ String getEnglishWord()
+ String getFinnishWord()
```

Then, write a main program called **ObjectsWordPairsProgram**. In the program, create three objects from the *WordPair* class and assign their references to reference variables. See the example output for the English and Finnish words.

Finally, the program should print the words held by the *WordPair* objects as shown in the example output. Use the *get methods* to get the English and Finnish words from the *WordPair* objects.

Example output:

```
school = koulu
student = opiskelija
textbook = oppikirja
```

Objects Song Array

Write a program called **ObjectsSongArray**. The program is a variation of the ***Objects Song Program*** exercise.

Reuse the **Song** class (no modifications should be needed) from the previous exercise and create an array of three *Songs*. Then create three ***Song*** objects and save their references to the array. See the example output for the song names etc. You can either use an array initializer or create the *Song* objects separately.

Finally, the program should iterate the array and print a song list exactly as shown in the example output. This time, use the ***getters*** to get data from the Song objects.

* Hint: To make it easier, you can use the *enhanced for loop* to iterate the array.

Example output:

```
=== List of songs ===
Adele: Easy on Me (2021)
Ed Sheeran: Shivers (2021)
Larkin Poe: Holy Ghost Fire (2020)
```

Objects Word Pairs Array

Write a program called **ObjectsWordPairsArray**. This exercise is a variation of the *Objects Word Pair Program* exercise.

Please reuse the **WordPair** class from the previous exercise and create an array of three *WordPairs*. Then create three *WordPair* objects and save their references to the array. See the example output for the English and Finnish words. You can either use an array initializer or create the *WordPair* objects separately.

Finally, the program should iterate the array and print the words held by the *WordPair* objects as shown in the example output.

Example output:

```
school = koulu
student = opiskelija
textbook = oppikirja
```

Objects Certification Program

First, write a new class called **Coach**. The features of the class should be exactly as shown in the diagram below.

```
Coach
- String name
- int score
+ Coach(String name, int points)
+ String getName()
+ int getScore()
```

Then, write a main program called **ObjectsCertificationProgram**. In the program,

1. Create an array of four *Coaches.*
2. Input names and scores of four coaches from the user. Create a Coach object for each coach and assign its reference to the array.
3. Input the minimum passing score.
4. List all those coaches (name, score) who have passed the test.
5. Finally, show the certification test passing rate.

Example output:

```
Enter coach name: John Doe
Enter John Doe's score: 42
Enter coach name: April Sun
Enter April Sun's score: 50
Enter coach name: Harry Lime
Enter Harry Lime's score: 10
Enter coach name: William Berry
Enter William Berry's score: 48

Enter the minimum passing score: 40

Passing scores
John Doe (42 points)
April Sun (50 points)
William Berry (48 points)
The passing rate is 75.0 %
```

Objects Clock Time Program

This time, we want to represent robust clock times. In the real world, a valid clock time is between **00:00** and **23:59**

Write a new class called **ClockTime** that represents clock time.

ClockTime
- int hours - int minutes
+ ClockTime() + void add(int hours, int minutes) + public String toString()

The additional business rules are the following:

- Initially, the clock time is always **00:00**
- The ***add*** method ***rolls the time forward*** by the given number of hours and minutes. If either parameter value is negative, the time should not change.
- The ***toString*** method returns the textual representation of the clock time in the "hh:mm" format Hint: You can use if statements to determine if you need to add a leading zeros.

NOTE! Please use the ready-made program class below to test your **ClockTime** class. You should not modify the program class at all.

ObjectsClockTimeProgram.java

```java
import java.util.Scanner;

public class ObjectsClockTimeProgram {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        ClockTime clockTime = new ClockTime();
```

```
        System.out.print("Enter hours to add: ");
        int hours = Integer.parseInt(input.nextLine());

        while (hours >= 0) {
            System.out.print("Enter minutes to add: ");
            int minutes = Integer.parseInt(input.nextLine());

            clockTime.add(hours, minutes);
            System.out.println(clockTime + "\n");

            System.out.print("Enter hours to add: ");
            hours = Integer.parseInt(input.nextLine());
        }

        input.close();
    }
}
```

Example output:

```
Enter hours to add: 11
Enter minutes to add: 30
11:30

Enter hours to add: 5
Enter minutes to add: 15
16:45

Enter hours to add: 10
Enter minutes to add: 55
03:40

Enter hours to add: 0
Enter minutes to add: 30
04:10

Enter hours to add: 0
Enter minutes to add: 1439
04:09

Enter hours to add: 47
Enter minutes to add: 1
03:10

Enter hours to add: -1
```

Objects Translator Array

Write a program called **ObjectsTranslatorArray** that inputs an English word from the user and translates it to Finnish. The program is a variation of your ***ArrayWords*** program.

This time, the program can translate only the following words: "bird", "bar", "bus", "car", "cat", "dog".

If the English word is unknown, the program should display "**Unknown word**".

- This time, use one array only. Reuse the **WordPair** class from a previous exercise and create an array of *WordPairs*.

Example output:

```
Enter an English word: car
auto
```